

■ Main code

Input: thickness array $\mathbf{t}$, refractive index array $\mathbf{n}$, Boundary $\mathcal{B}$, Target S_{target}

Output: optimized thickness array $\mathbf{t}^*$

```

1:  for each parameter  $t_i \in \mathbf{t}$  do
2:     $[L_i, U_i] \leftarrow \mathcal{B}(t_i)$ 
3:     $t_i \sim \mathcal{U}(L_i, U_i)$ 
4:  end for

5:  for  $i = 1$  to Max_Iterations do
6:    Initialize empty vector  $S_{pred}$ 

7:    for  $k = 1$  to Number_Of_EML do
8:      Initialize  $M_1, M_2, M_3, M_4 \leftarrow 0$ 

9:      for each direction  $dir \in \{Upward, Downward\}$  do
10:        Set index  $p \in \{0, 1\}$  and range based on  $dir$ 
12:        Update  $(M_{1...4}^{(p)})$  using  $f_{eq1...4}$  recursively.
13:      end for

14:       $(|\rho_1|, \rho_1, |\rho_2|, \rho_2, T) \leftarrow f_{eq5,6}(n_j, n_{env}, M_1, M_2, M_3, M_4)$ 
15:       $(TI, FP) \leftarrow f_{eq7,8}(|\rho_1|, \rho_1, |\rho_2|, \rho_2, T, n_j, t_j, t_{EMZ})$ 
16:       $S \leftarrow TI * FP * PL$ 
17:       $S_{pred} \leftarrow S_{pred} + S$ 
18:    end for

19:     $L_{MSE} \leftarrow \frac{1}{K} \sum_{k=1}^K (S_{target,k} - S_{pred,k})^2$ 
20:     $g \leftarrow \nabla_t L_{MSE}$  via backward-mode differentiation
21:     $t \leftarrow \text{AdamUpdate}(t, g)$ 
22:  end for

23: return  $\mathbf{t}$ 

```

■ Function Code (Forward)

Definitions:

- f : Physical function
- $f.gen$: The generation depth of node f
- $\mathbf{x}$: A set of input arguments

```

1: function Forward( $f, \mathbf{x}$ )
2:    $\mathbf{P} \leftarrow \{x \mid x \in \mathbf{x} \text{ and } x \text{ is output a function node}\}$ 
3:   if  $\mathbf{P}$  is not empty then
4:      $f.gen \leftarrow \max(\{p.gen \mid p \in \mathbf{P}\})$ 
5:   else
6:      $f.gen \leftarrow 1$ 
7:   end if

8:    $f.inputs \leftarrow \mathbf{x}$ 
9:    $f.outputs \leftarrow \text{Evaluate}(f, \mathbf{x})$ 
10:  return  $f.outputs$ 
11: end function

```

■ Function Code (Backward)

Definitions:

- F : A set of all functional nodes collected from the computation graph
- $f.gen$: The generation depth of node f
- G_{acc} : A dictionary to accumulate gradients for each variable
- g_{init} : The initial gradient

```

1: function Backward( $f_{loss}$ )
2:    $g_{init} \leftarrow 1$ 
3:    $F \leftarrow \text{CollectAllFunctions}(f_{loss})$ 
4:   return RunBackward( $F, g_{init}$ )
5: end function

6: function RunBackward( $F, g_{init}$ )
7:    $F_{sorted} \leftarrow \text{Sort } F \text{ by } gen \text{ in descending order}$ 
8:    $G_{acc} \leftarrow \emptyset$ 
9:    $g_{curr} \leftarrow g_{init}$ 
10:  for each  $f \in F_{sorted}$  do
11:     $\nabla_{local} \leftarrow \left( \frac{\partial f.output}{\partial f.inputs} \right) \cdot g_{curr}$ 
12:    for each variable  $v \in f.inputs$  do
13:       $G_{acc}[v] \leftarrow G_{acc}[v] + \nabla_{local,v}$ 
14:    end for

15:    if  $gen(next(f)) < f.gen$  then
16:       $g_{curr} \leftarrow G_{acc}[next(f).output]$ 
17:      Reset  $G_{acc}$  for the next level
18:    end if
19:  end for
20:  return  $G_{acc}$ 
21: end function

```